深層学習の基礎理論 (1)—本体資料 P1∼P8
２０２２年１０月１９日分

高山 幸秀
立命館大学理工学部数理科学科

Abstract. １９９０年代に一度は下火になったニューラルネットが、近年、多層ネットによるディープラーニング（以下、深層学習）として復活し、AI の新技術として爆発的発展を遂げた。現在、さまざまなアプリケーションの開発が盛んに行われているが、技術的限界も次第に明らかになってきており、深層学習技術を補完、あるいはこれを超える全く新しい技術の提案も始まっている。このような状況において、今一度ニューラルネットや深層学習の基本原理を押さえておくこは意味のあることと考えられる。本講座では、主にこの方面を専攻していない学生諸君を念頭に、深層学習の基礎理論うち、入門的な部分について講述する。

Contents

1. はじめに 2
1.1. この資料の使い方 2
1.2. 自己紹介 2
2. 順伝播型ネットワーク（多層パーセプトロン) 3
2.1. ユニット 3
2.2. 多層パーセプトロン (or 順伝播型ネットワーク 4
2.3. 活性化関数 5
2.4. Example 5
2.5. 数学的記述 5
2.6. Question 6
2.7. 多重パーセプトロンの学習 6
3. 次回・次々回予告 7
3.1. １１月８日 7
3.2. １１月２２日 7

Date: 2022 年 10 月 19 日、11 月 8 日、11 月 22 日、徳島大学工学部集中講義.

1. はじめに

1.1. この資料の使い方. この資料とは別に、手書きノートをスキャンして PDF 化した資料があると思いますが、それが「本体資料」です。しかしこれは、あくまで教室で講義する時のメモとして書かれたものですので、説明がほとんどありません。

そこで、この資料では教室での講義で口頭で説明しているような事を書きます。「本体資料」を参照しながら、読んでください。

1.2. 自己紹介. 私は１９８５年から１９９６年ぐらいまで、数理論理学的アプローチでの人工知能研究を行っていましたが、その後、計算機科学から純粋数学に転じました。ニューラルネットに関しては、１９９０年代に私の研究室の学生の中に卒業研究のテーマとして研究したいという人がちらほら現れ、それに付き合っていた程度です。ですから、私はニューラルネットの専門家ではありません。

しかしながら、私の人工知能研究者時代の終わり頃に現れたニューラルネットを見て、「こんな代物でまともな人工知能ができるわけない」と思っていたものが、１０数年後突如脚光を浴びて、あれよあれよという間に実用化が進む様子を目の当たりにし、「これはいったい何が起こったのか？」と思って、数学研究の合間に少し勉強してみた次第で、その結果がこの集中講義です。

この資料の中でも、随所に「こんなんで、何で上手く行くんや？」という素人的偏見に基づく突っ込みが書かれていますが、それはそういうものとして受け止めておいてください。

2. 順伝播型ネットワーク（多層パーセプトロン)

- １９８０年代は、「論理的推論が行える計算機を開発すれば人工知能が実現できるのでは？」という発想に基づき、知識情報処理のアプローチでの人工知能研究が盛んに行われていました。多くの計算機科学者たちが数理論理学に興味を持って、それを計算機技術に生かせないか？ と研究を続けていました。しかし 、なかなか実用化は難しいということで１９９０年代に入ってからは、次第に下火になって行きました。

- 知識情報処理アプローチに代わるものとして、１９９０年代に入ってニューラルネットとファジー制御が注目され始めました。ファジー制御はファジー論理と呼ばれる論理学の一種を使って柔軟な推論が行えるようにしようというアイディアですが、ニューラルネットは論理学とは別のアイディアです。

- 人間の脳は、膨大な数の神経細胞が繋がってネットワークを構成（神経系）し、その中で電気信号をやりとりしながら情報処理をしているらしく、それを真似たら人工知能が出来るのではないか？ それがニューラルネット技術の元々のアイディアだったようです。アイディア自身は古くからあったようですが、郵便番号の手書き文字認識への応用が成功して、注目されるようになったようです。

- しかしこの技術も間もなく困難にぶつかり、その解決法がなかなかみつからないまま、２１世紀に入る前にニューラルネットの研究者のほとんどが撤退してしまい、２１世紀の初めのブレークスルーまで休眠状態に陥ります。

- この節では、人間の脳の神経系をどのようにモデル化したのか？ そして技術的にどこで壁にぶつかったのか？ を見ていきます。

2.1. **ユニット．** ユニットとは１つの神経細胞をモデル化したものです。神経細胞は他の神経細胞から信号を受け取り、それに反応して自ら何らかの信号を発生させ、それを他の神経細胞に伝えます。

- 丸い玉として描かれているものが、神経細胞本体をモデル化したものです。これをユニットと呼びます

- 左の方に複数の、そして右の方に１本の線が伸びていますが、左の方が他の神経細胞から信号を受け取るための回線で、右の方が自分が発生させた信号を他の神経細胞に伝えるための回線です。

- 他の神経細胞からの入力信号を $x_1, x_2, \ldots$ と表しています。自分が発した出力信号は z で表しています。

- 他の神経細胞からの入力信号は、平等に扱うとは限らず、それぞれに「重み付け」をして受け取ります。その「重み付け」を $w_1, w_2, \ldots$ と表しています。そして、入力信号を全部集めたもの $\sum_i w_i x_i$ を考えます。

- さらに、入力信号を集めたもの $\sum_i w_i x_i$ に、多少「下駄を掃かせ」たりもします。それをバイアスと言って b で表しています。入力信号を全部集めたものに下駄を

掃かせたものを $u = \sum_i w_i x_k + b$ と表しています。

- 神経細胞は他の神経細胞から信号を受け取った後、それに「反応して」自ら信号を発生させるのですが、それは受け取った入力信号 u から出力信号 z への関数と見ることができます。活性化関数と呼び、f で表しています。つまり（神経細胞をモデル化した）ユニットは $u \overset{f}{\longmapsto} z$ なる関数と考えることができます。

- 活性化関数 f が凄い計算処理をしていれば、1 個の神経細胞自体が凄い知能を持っていることになります。しかし、後で示すように活性化関数は案外単純な関数で、こんなものを組み合わせて何で高度な知能が実現できるのか不思議なぐらいです。しかし人間の脳もそんな風に単純な信号処理装置が大量に、かつ、複雑に絡み合うことによって、高度な知能を実現しているのでしょう。

2.2. 多層パーセプトロン (or 順伝播型ネットワーク.

- ここから先は、本物の脳の神経細胞系よりもうんと単純なものだと思われますが、ユニットをいくつか集めたものを「層」と呼び、それを何層も積み重ねたものを「多重パーセプトロン」（または「多層ネット）と呼びます。

- 図にあるように、個々のユニットが完全に独立して他ユニットと信号のやりとりをするのではなく、ユニットがいくつか集まった層単位で信号のやりとりをします。外部から入ってきた信号は、まず第 1 層が受け取り、信号を発生ささせ第 2 層に送り etc. といったことを繰り返して、最終層を経て出力信号を出力します。

- 第 k 層と第 $k+1$ 層の間の信号のやりとりが、拡大図で詳しく示されています。例えば、第 $k+1$ 層の一番上のユニットは、第 k 層の複数のユニットから信号を受け取りますが、その重みづけは $w_{11}, w_{12}, \ldots$ と表されています。第 $k+1$ 層の
  - 一番上のユニットに第 k 層から信号 $x_1, x_2, \ldots, x_I$ がそれぞれ重み付け $w_{11}.w_{12}, \ldots, w_{1_I}$ で入ってきて、さらにバイアス b_1 が加えられて $u_1 = \sum_i w_{1i} x_i + b_1$ がこのユニットの入力になります。
  - 二番目のユニットに第 k 層から信号 $x_1, x_2, \ldots, x_I$ がそれぞれ重み付け $w_{21}.w_{22}, \ldots, w_{2_I}$ で入ってきて、さらにバイアス b_2 が加えられて $u_2 = \sum_i w_{2i} x_i + b_2$ がこのユニットの入力になります。
  - 同じ要領で、上から第 j 番目のユニットに対する入力は $u_j = \sum_i w_{ji} x_i + b_j$ となります。i, j の添え字に注意しましょう。

- 以上を纏めて、第 k 層から出力された信号 $x_1, x_2, \ldots,$ を第 $k+1$ が受けっとって、第 $k+2$ 層へ向けての信号を発生させる様子は、ベクトルと行列を使ってアフィン変換 $\bar{z} = \bar{w} \cdot \bar{x} + \bar{b}$ で表されることになります。

- 以上のことから、ニューラルネットシステムを正確に書き表そうとすれば、線形代数（ベクトル、行列、線形写像、アフィン写像）を駆使しなければならないことがわかります。

- また、上で見た重み付けデータを集めた行列 $\overline{w}$ とバイアスデータを集めたベクトル $\bar{b}$ の２つが、個々の多層パーセプトロンのパラメータになっているわけです。全ての層の間のパラメータを集めるとう、膨大な数になります。

2.3. 活性化関数.

- 活性化関数として使われているものを色々挙げました。基本的に単調増加関数です。つまり各ユニットは、強い信号を入力すれば強い信号を出力するという風にふるまいます。しかし、グラフを見てみると入力 u の絶対値が大きくなると、関数値の増減が緩やかになっています。これは生物の実際の神経系の反応に近い形だとされています。つまり、いきなり強い刺激が来ても、それに対して過剰には反応しないという神経系の性質をモデル化しているようです。

- logistic sigmoid function, 双曲線正接関数のような非線形関数（一次関数でないもの）の方が、より実際の神経細胞の反応に近いのだそうで、ニューラルネットとして実装した場合も良い性能が期待できるそうですが、正規化線形関数、線形関数、「名もなき関数」のようなものの方が実装が簡単で計算効率も良いので、実際にはよく採用されるようです。

- どういう活性化関数を使えば、深層学習システムとしての性能がどのように良くなるのかについては確たる理論はなく、アプリケーション毎に試行錯誤してアドホックにやっていると思われます。

2.4. **Example.**

- 以後の話で、理論内容を例示するために、３層パーセプトロン (３層ネット）の具体例を示しています。

- 第１層から第２層への信号のやり取りは $\bar{x} \mapsto \bar{u}^{(2)}$ と表され、第２層は行列 $\overline{w}^{(2)}$ とバイアス $\bar{b}^{(2)}$ を使ったアフィン変換になっています。第２層の出力は入力データである $\bar{u}^{(2)}$ の各要素に活性化関数を作用させて作られています。第２層から第３層の信号のやり取りも、行列 $\overline{w}^{(3)}$ とバイアス $\bar{b}^{(3)}$ を使ったアフィン変換で書き表されています。

- 第２層の各ユニットでは、全て同じ活性化関数 f が使われています。第３層は出力層ですが、ここでも活性化関数 f が使われています。

- 少なくとも１つの層では同じ活性化関数が使われていますが、これはシステムの構成を単純化するためであり、また、ユニット毎に活性化関数を変えるメリットがよくわからないからだと思います。

2.5. 数学的記述.

- ３層パーセプトロンの Example からもわかるように、入力データ $\bar{x}$ が３層パーセプトロンで処理され、出力データ $\bar{z}^{(3)} = [y_1, y_2]$ となるまでには、各層の間のデータの重み付けを表す行列 $\overline{w}^{(k)}$ やバイアス $\bar{b}^{(k)}$ の値（と活性化関数）が関係していて、これらの行列やベクトルの値を変えれば出力も変わってきます。

- つまりニューラルネットワークの性能は、入力 $\bar{x}$ に対して「望ましい」出力 $\bar{z}$ を返せるように、いかにして $\bar{w}^{(k)}$ や $\bar{b}^{(k)}$ の値 (パラメータ) をチューニングするかによって決まるわけです。

2.6. **Question.**

- パラメータのチューニングは、ニューラルネットに「学習させる」ことによって行います。すなわち、「お手本を沢山見せてやるから、それを見て自分で会得せよ」という方法をとります。

- 結局、ニューラルネットの研究が２０世紀末に一度「ほとんど絶滅」してしまったり、２１世紀に入って復活したのも、全ては「ニューラルネットに上手に学習させる方法」が長い間分からなかったからです。Example では 3 層の例を挙げましたが、層の段数を上げないと、なかなか性能が出ないことが経験的に知られていて、しかしながら、層の段数が上がると、なかなかうまく学習させられないことがネックになっていたようです。

- ２１世紀に入ってからの「復活」は、段数の多い層 (多層パーセプトロン) に対してもうまく学習させられる方法が発見されたからだと言われています。

2.7. **多重パーセプトロンの学習.**

- 多層パーセプトロンの「学習」は、「訓練データ (or 教師データ)」（望ましい入力と出力のデータ）を持ってきて、「訓練データ」の入力データを多重パーセプトロンに食わせて、実際に出てきた出力と「訓練データ」の出力データと比較して、その差を「誤差関数」と呼ばれるもので適当に数学的処理をして、誤差の大きさを評価することが第一歩です。

- 誤差関数で誤差の大きさを評価したら、次は重み付けやバイアスといったパラメータの調整、つまり再設定をする必要がありますが、それについては次回「確率的勾配降下法」のところで述べます。

- 誤差関数はアプリケーションによって色々なものが使われるようですが、ここでは代表的なものを挙げています。
  (1) 「回帰問題」とは、それ以外の場合で、例えば画像検索や昔の白黒写真をカラーで再生するといったシステムの場合です。ここで、訓練データの個々の $(\bar{x}_i, \bar{d}_i)$ は $\bar{x}_i$ が入力データで、$\bar{d}_i$ が出力データ（として望ましいもの）です。

  (2) 「二値問題」とは、顔認識のような Yes/No 型の出力をするシステムの場合です。また、「多クラス問題」とは、手書き文字認識のような有限個のパターンのうちのどれか？ を判定させるシステムの場合です。いずれの問題も、連続値を持つ出力データをどこで切って Yes/No, あるいは有限個のクラスに振り分けるかを判定しなければならないので、確率論を使って誤差関数を構成しています。

  (3) 「二価問題」で $P(d|\bar{x})$ は所謂「条件つき確率」を表す式で、$\bar{x}$ という入力信号が入ってきた状況において、多層パーセプトロンが出力信号 $d = 0$ または $d = 1$ を出力する確率を表します。これが「多クラス問題」の場合は $P(C_i|\bar{x})$,

但し、C_i は複数あるクラスの名前、という条件付き確率を使って、かなり複雑なことをやって誤差関数を決めています。

3. 次回・次々回予告

さて、多層パーセプトロンの学習には、訓練データと誤差関数を使うことを学びましたが、それでは、これらを使って、どうやって重み付けやバイアスといったパラメータをチューニングすれば良いのでしょうか？ 次回以降はその問題を考えていきます。

3.1. １１月８日.

(1) 確率的勾配降下法
- 勾配降下法とその問題点
- 確率的勾配降下法

(2) 誤差逆伝播法
- 勾配計算は何処が難しいのか？
- 誤差逆伝播法による計算

3.2. １１月２２日.

(1) 誤差逆伝播法 (つづき)
- 参考 1
- 参考 2
- 勾配消失問題/解決法

(2) 自己符号化器
- 自己符号化器
- ディープネットの事前学習

Yukihide Takayama, Department of Mathematical Sciences, Ritsumeikan University, 1-1-1 Nojihigashi, Kusatsu, Shiga 525-8577, Japan
Email address: takayama@se.ritsumei.ac.jp